

01

HACKATHON ETHEREUM LIMA 2026 · TRACK ARBITRUM

BenevrIA

Una IA gratuita para todos, cuyo **nivel de modelo lo decide un contrato inteligente** según el conocimiento que la comunidad le enseña.

CATEGORÍA

IA y Tecnologías Emergentes

RED

Arbitrum Sepolia

CONTRATO

Rust · Stylus

EQUIPO

nadevrix

02

EL PROBLEMA

Se entrenaron con nuestro conocimiento y nos lo cobran de vuelta

Hay conocimiento que la IA no tiene

El trámite que solo sabe quien lo hizo. La jerga del oficio. El procedimiento que ningún manual escribió. Eso los modelos lo alucinan — y quien lo sabe no recibe nada por saberlo.

Y el acceso se paga

Quien no puede pagar una suscripción mensual queda fuera. En Bolivia, veinte dólares al mes no es una abstracción.

Si no puedes nombrar a la persona que lo usaría mañana, no tienes un problema real. La nuestra: *el profesional que sabe lo que la IA inventa mal, y hoy no cobra por ello.*

03 LA SOLUCIÓN

Aportas conocimiento, sube el nivel de todos, cobra quien enseñó

1

Enseñas

Escribes conocimiento que los modelos grandes suelen inventar.

2

El contrato verifica

Compara tu aporte contra todo el corpus. Si es un reenvío de algo que ya está, lo rechaza. Si es nuevo, emite puntos.

3

Sube el nivel de todos

La novedad acumulada desbloquea modelos mejores. Los usa cualquiera, haya aportado o no. Ese es el bien común.

4

Cobras

Cuando entra dinero, se reparte en proporción exacta a la novedad que aportaste.

El nivel es colectivo

Todos usan el modelo que la comunidad ganó, incluso quien nunca aportó.

El dinero es individual

Solo cobra quien aportó, en proporción exacta a su novedad verificada.

04 LA PREGUNTA DEL JURADO

¿Y esto no funcionaría con una base de datos?

Un servidor puede calcular exactamente lo mismo. Lo que no puede es **probarlo**.

Estas tres decisiones viven on-chain porque son justo aquellas donde el operador tendría incentivo a mentir:

DECISIÓN	SI LA TOMARA UN SERVIDOR
¿Tu aporte es novedoso?	“Es duplicado, no cobras” — y no tienes cómo comprobarlo
¿Qué modelo corre?	“Bajó de nivel” mientras se embolsa la diferencia
¿Cuánto le toca a cada quien?	Un reparto que nadie puede auditar

05

IMPLEMENTACIÓN TÉCNICA

Por qué Stylus y no Solidity

El ataque económicamente racional contra un sistema de “aporta y gana” no es escribir basura —eso lo filtra un `if`— sino **reenviar conocimiento que ya está en el corpus, ligeramente reformulado**.

Detectarlo exige comparar el vector del aporte nuevo contra todo lo almacenado:

DIMENSIONES

64

int8, dos palabras de storage

VENTANA DEL CORPUS

256

vectores comparados

OPERACIONES

~16.000

mult-sumas por transacción

“Usamos Stylus porque hacer esto en Solidity sería *impagable en gas*. En WASM compilado a código nativo es aritmética de registro.”

Medido, no supuesto. Un mismo trámite escrito de tres formas contra el original: reenvío casi idéntico **99,3 %** (rechazado), reescrito con otras palabras **67,5 %** (aceptado), tema distinto **22,1 %** (aceptado).

Qué corre dónde, y por qué

El modelo de IA **no vive en el contrato** — no cabe, y fingir lo contrario sería mentir. Lo que vive dentro es la **verificación**: la IA produce el dato, el contrato lo juzga.

```

flowchart TB
    A["Contribuyente  
escribe conocimiento"] --> B["Heurísticas + filtro de  
datos personales"]
    B --> C["Embedding  
64 dims · int8"]
    C --> D{"¿Hash ya visto?"}
    D -->|sí| X1["Rechazado"]
    D -->|no| E["Similitud coseno  
contra el corpus"]
    E --> F{"¿Similitud > 0,90?"}
    F -->|sí| X2["Rechazado:  
reenvío parafraseado"]
    F -->|no| G["Aceptado  
puntos = novedad"]
    G --> H["Puntaje colectivo"]
    H --> I{"Nivel = mín(konocimiento, tesorería)"}
    I --> J["El chat usa ese modelo"]
    K["Empresa compra créditos"] --> L["Tesorería"]
    L -->|30%| M["Pozo de aportantes"]
    L -->|70%| N["Presupuesto de inferencia"]
    M --> O["Cada quien reclama  
su proporción"]
  
```

Fuera de la cadena · barato

Heurísticas, filtrado de datos personales y cálculo del embedding.

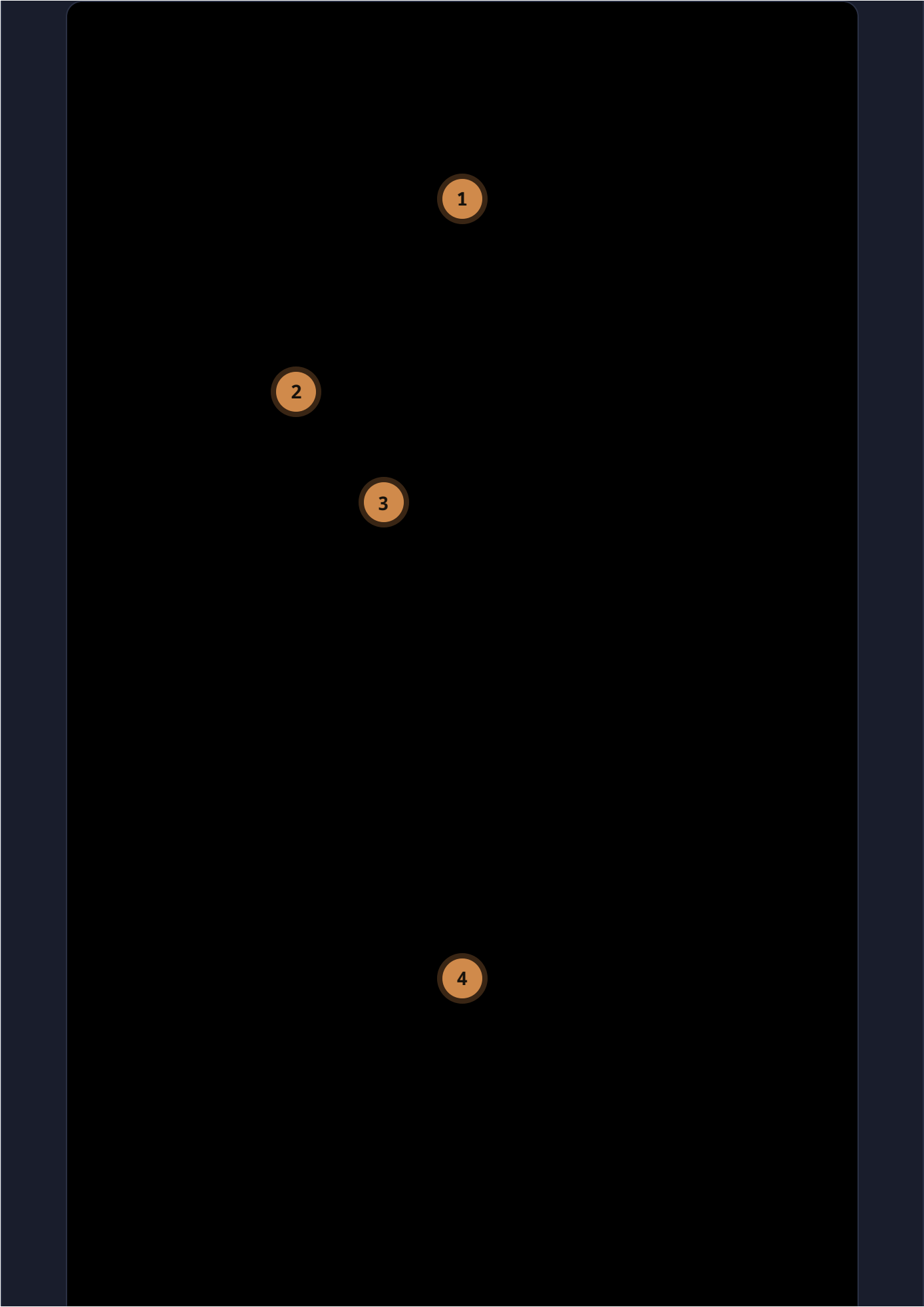
Dentro de la cadena · caro pero necesario

La comparación que decide si el aporte es nuevo — y por tanto si se cobra.

07

LA APLICACIÓN · PANTALLA DE INICIO

Lo primero que ve quien llega



ARBITRUM STYLUS · ETH LIMA 2026

Una IA gratis que la comunidad enseña

Las IAs se entrenaron con el conocimiento de todos y después nos lo cobran.
BenevrIA le da la vuelta: aportas lo que sabes, un contrato verifica que sea nuevo, y eso sube el nivel del modelo *que todos usan*. Cuando entra dinero, cobra quien enseñó.

Usar la IA gratis

Enseñarle algo

NIVEL ACTUAL

Base

Ling 3.0 Tiny

CORPUS VERIFICADO

9

aportes únicos

PUNTAJE COLECTIVO

44265

novedad acumulada

TEMAS PEDIDOS

4

por la comunidad

El problema

Hay conocimiento que la IA no tiene

El trámite que solo sabe quien lo hizo. La jerga del oficio. El procedimiento que ningún manual escribió. Eso los modelos lo alucinan — y quien lo sabe no recibe nada por saberlo.

Y el acceso se paga

Se entrenaron con el conocimiento de la gente y lo devuelven por suscripción. Quien no puede pagarla, queda fuera.

Cómo funciona

1

Enseñas

Escribes conocimiento que los modelos grandes suelen inventar: un trámite local, un procedimiento de tu oficio.

2

El contrato verifica

Compara tu aporte contra todo el corpus. Si es un reenvío de algo que ya está, lo rechaza. Si es nuevo, emite puntos.

3

Sube el nivel de todos

La novedad acumulada desbloquea modelos mejores. Los usa cualquiera, haya aportado o no. Ese es el bien común.

4

Cobras

Cuando entra dinero, se reparte en proporción exacta a la novedad que aportaste. Sin lista de beneficiarios ni aprobaciones.

¿Y esto no funcionaría con una base de datos?

Un servidor puede calcular exactamente lo mismo. Lo que no puede es **probarlo**. Estas tres decisiones son justo aquellas donde el operador tendría incentivo a mentir:

Decisión	Si la tomara un servidor
¿Tu aporte es novedoso?	"Es duplicado, no cobras" — y no tienes cómo comprobarlo
¿Qué modelo corre?	"Bajó de nivel" mientras se embolsa la diferencia
¿Cuánto le toca a cada quien?	Un reparto que nadie puede auditar

Por eso viven en un contrato **Stylus**: la comparación contra el corpus son ~16.000 multiplicaciones-sumas por transacción. En Solidity sería impagable en gas; en WASM compilado a código nativo es aritmética de registro.

Empieza por donde quieras

Pedir un tema

Dinos qué no sabe la IA y alguien se lo enseñará.

Ver el panel

Nivel, corpus y tesorería, leídos de la cadena.

Cobrar

Lo que te toca por lo que enseñaste.

- 1 **La propuesta en una frase.** Qué es y por qué existe, sin jerga.
- 2 **Usar la IA gratis.** Lleva al chat. No pide wallet: es gratis para todos.
- 3 **Cifras en vivo** leídas del contrato: nivel, corpus, puntaje y temas.
- 4 **Los cuatro pasos** del ciclo: enseñar, verificar, subir de nivel, cobrar.

08

LA APLICACIÓN · PANEL

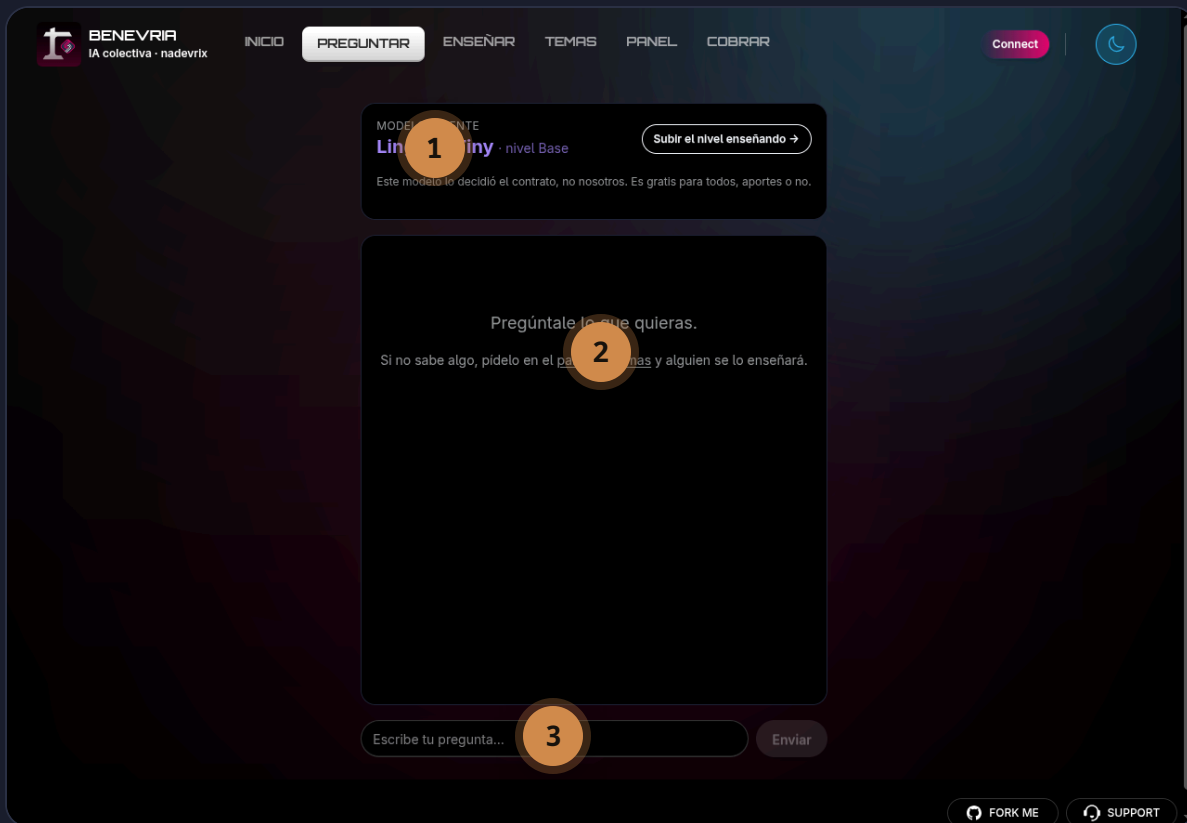
El estado del organismo, leído de la cadena

- 1 Nivel vigente y modelo.** Lo decidió el contrato, no el operador.
- 2 Progreso al siguiente nivel.** Cuántos puntos faltan, en tiempo real.
- 3 Qué frena el nivel.** El recuadro marcado señala el cuello de botella: aquí falta *conocimiento*, no dinero.
- 4 Corpus, puntaje y tesorería.** Todo verificable.
- 5 Prueba de procedencia.** Época, bloque real de L2 vía ArbSys y enlace al contrato en Arbiscan.

09

LA APLICACIÓN · PREGUNTAR

La IA, gratis y sin conectar nada



- 1 **Modelo vigente.** La página no lo elige: se lo pregunta al contrato.
- 2 **Conversación.** Abierta a cualquiera, aporte o no — es la tesis del proyecto.
- 3 **Sin wallet, sin cuenta, sin correo.** Ponerle un muro contradiría la idea.

Al subir de nivel, este mismo recuadro pasa a mostrar un modelo mejor. Es el momento a capturar en el video.

10

LA APLICACIÓN · ENSEÑAR

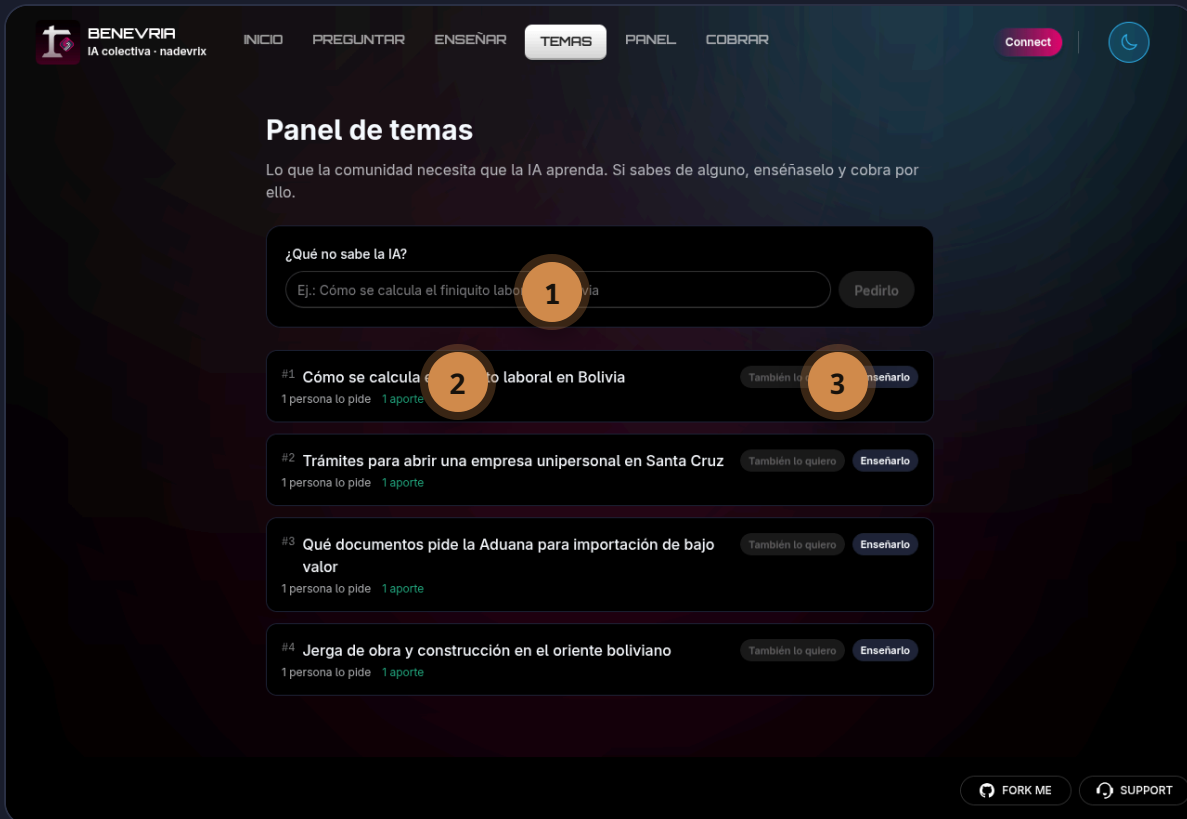
Las dos capas, visibles en la interfaz

- 1 Tu aporte.** Un trámite local, un procedimiento del oficio, jerga técnica.
- 2 Paso 1 — fuera de la cadena.** Heurísticas, limpieza de datos personales y embedding. Barato: filtra sin gastar transacción.
- 3 Tema.** Se puede responder a una petición concreta del panel.
- 4 Paso 2 — dentro de la cadena.** Aquí el contrato compara y decide. Es la única parte que necesita ser confiable sin confiar en nadie.

11

LA APLICACIÓN · PANEL DE TEMAS

La comunidad dice qué le falta aprender

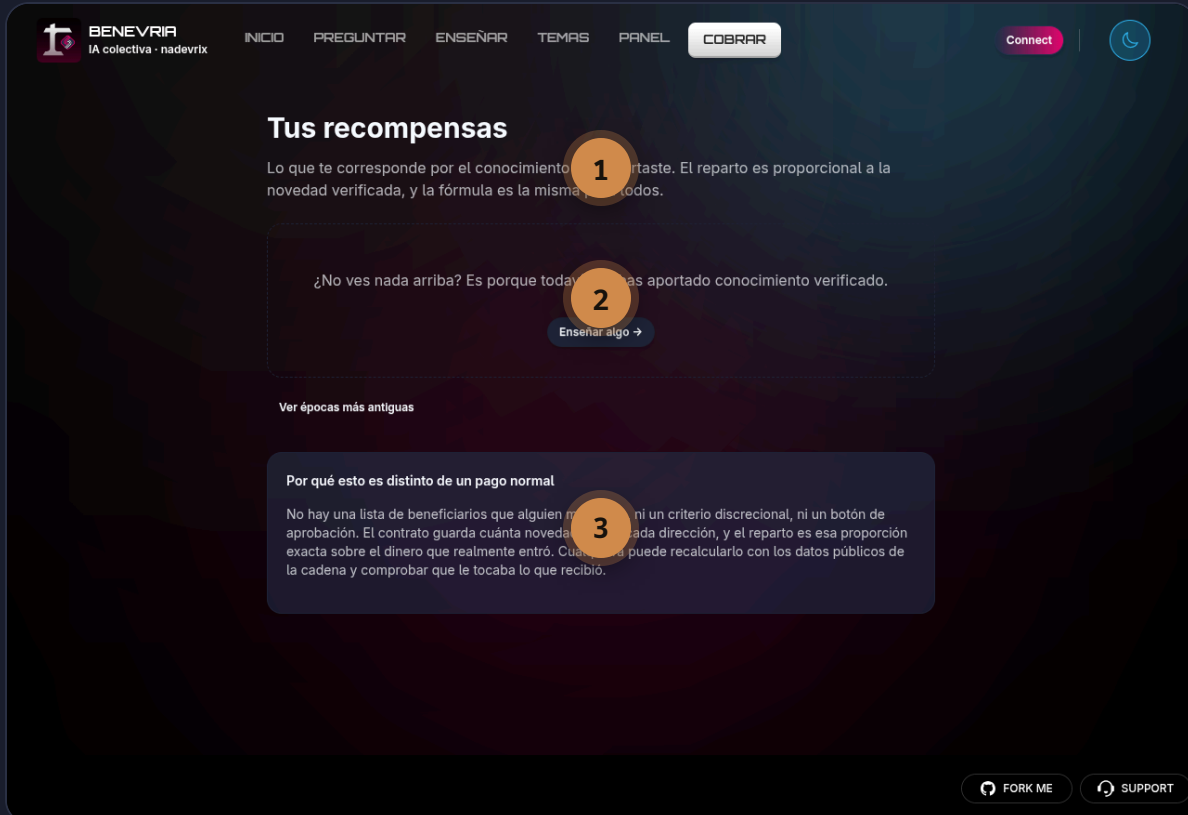


- 1 **Pedir un tema.** Resuelve el problema de “¿qué aporte?”: no se adivina, se responde una demanda concreta.
- 2 **Demanda medida.** Cuántas personas lo piden y cuántos aportes recibió.
- 3 **Votar o enseñar.** Un voto por dirección, registrado en cadena: nadie infla la demanda desde un servidor.

12

LA APLICACIÓN · COBRAR

El reparto es una fórmula pública, no una decisión



- 1 **Proporcional a la novedad verificada**, con la misma fórmula para todos.
- 2 **Por época**. Cada fila muestra el cálculo entero: `pozo × puntos ÷ total`.
- 3 **Sin lista de beneficiarios ni botón de aprobación**. Cualquiera puede recalcularlo desde los datos públicos de la cadena.

13

VERIFICABLE AHORA MISMO

Desplegado y funcionando en Arbitrum Sepolia

CORPUS

9

aportes verificados

PUNTAJE

44.265

novedad acumulada

TEMAS

4

pedidos por la comunidad

TESORERÍA

0,056

ETH para inferencia

CONTRATO	0xdf48b19ad2c77050fe08fef0dde577f4e5066e6d
RED	Arbitrum Sepolia · chainId 421614
TESTS	28 / 28 pasan · cargo stylus check OK · 32,6 KB
BLOQUE L2	297.357.776

Prueba de que ArbSys funciona en producción. El bloque de L2 va por 297 millones mientras el de L1 Sepolia va por 11,4 millones. La llamada al

precompilado `0x64` resuelve de verdad — en Arbitrum `block.number` devuelve el bloque de L1, y usarlo daría un reloj equivocado.

14

USO DEL ECOSISTEMA ARBITRUM

Cuatro piezas, cada una por una razón

PIEZA	PARA QUÉ
Stylus	Todo el núcleo: matemática vectorial en punto fijo, imposible de costear en Solidity
ArbSys (0x64)	Bloque real de L2 para las épocas de reparto
Arbitrum Sepolia	Red de despliegue
x402	Pago de inferencia en USDC desde la tesorería. El facilitador de PayAI soporta <code>arbitrum-sepolia</code>

15

LO QUE TODAVÍA NO ES

Límites declarados, no disimulados

Todo el circuito funciona en testnet: contrato, verificación, niveles, tesorería y reparto son reales. Lo único simulado es el valor del dinero.

El último tramo del pago aún no es autónomo

OpenRouter todavía no acepta x402 — verificado contra su propia API. Hasta que complete su migración, el pago al proveedor pasa por una credencial. El protocolo está listo; falta el proveedor.

El embedding local es léxico, no semántico

Detecta reenvíos y copias, no equivalencia con vocabulario distinto. Con un modelo de embeddings real, el segundo caso también se detecta y el contrato no cambia.

El filtro de datos personales es un mejor esfuerzo

Ningún filtro es infalible, y prometerlo sería exposición legal gratuita.

La ventana del corpus es de 256 vectores

Sin tope, el costo de aportar crecería sin límite. El barrido histórico completo es trabajo fuera de la cadena.

16

SIGUIENTE PASO

Qué sigue

x402 directo

Cuando el proveedor lo active, la tesorería paga sola y el circuito cierra sin intermediarios.

Disputas optimistas

Ventana de desafío con stake — el mismo patrón con el que Arbitrum valida su propio estado.

Corpus por dominio

Segmentos verificados por área: legal, técnico, oficios. Los segmentos valen más que el bulto.

El mecanismo corre solo: nadie aprueba pagos, nadie decide quién cobró, nadie puede desviar fondos. Lo que falta no es tecnología — *es demanda real*.

17

QUIÉN LO HIZO

Entrega individual



Rodrigo Ricaldez Martinez

NADEVRIX · SANTA CRUZ, BOLIVIA

Contrato en Rust sobre Stylus, frontend, despliegue y documentación. Todo el proyecto, de punta a punta, por una sola persona.

Contrato

Rust · Stylus SDK · 28 tests · verificación vectorial on-chain

Aplicación

Next.js · wagmi · viem · integración con modelos de pesos abiertos

Experiencia previa en desarrollo de contratos on-chain en Rust, que es lo que hizo viable apuntar directo a Stylus en el tiempo de un hackathon.

WEB nadevrix.dpdns.org

GITHUB github.com/nadevrix

LINKEDIN in/nadevrix

X x.com/nadevrix

TELEGRAM [@nadevrix](https://t.me/nadevrix)

CORREO ricaldezmartinezrodrigo12@gmail.com

BENEVRIA · NADEVRIX · ETH LIMA 2026